

Prova del 08-Set-2022 (90 minuti)

Cognome _____ **Nome:** _____

[illegible][illegible]

4) Utilizzando NumPy e OpenCV, implementare in Python la funzione *esercizio(img)* che riceve un'immagine a colori *img* (con tre byte per pixel) e deve eseguire le seguenti operazioni:

- 1) Convertire *img* in grayscale memorizzando il risultato in un'immagine chiamata *gray_img*.
- 2) Applicare due filtri gaussiani a *gray_img*: il primo con dimensione 5×5 e $\sigma=1$, il secondo con dimensione 11×11 e $\sigma=4$. Siano *img_g1* e *img_g2*, rispettivamente, le due immagini ottenute applicando tali filtri.
- 3) Calcolare la differenza pixel-a-pixel sottraendo *img_g2* da *img_g1*, memorizzando il risultato in un'immagine con un intero a 16 bit per ogni pixel (in modo da ottenere sia i valori negativi che quelli positivi), quindi conservare solo i valori positivi (ponendo a zero tutti gli altri), normalizzarli fra 0 e 255 e memorizzare il risultato in un'immagine di byte *img_d*.
- 4) Eseguire l'algoritmo di Canny, senza effettuare il primo passo (smooth Gaussiano), ma partendo direttamente dall'immagine *img_g2*; come soglie per l'isteresi utilizzare 30 e 70.
- 5) Restituire un'immagine a colori in cui, per ciascun pixel, la componente blu è il valore corrispondente in *gray_img*, la componente verde è il valore corrispondente ottenuto con l'algoritmo di Canny e la componente rossa è il valore corrispondente in *img_d*.

```
import numpy as np
import cv2 as cv

def esercizio(img):
    gray_img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    img_g1 = cv.GaussianBlur(gray_img, (5,5), 1)
    img_g2 = cv.GaussianBlur(gray_img, (11,11), 4)
    diff = img_g1.astype(np.int16) - img_g2
    img_d = np.clip(diff, 0, 255).astype(np.uint8)
    img_d = cv.normalize(img_d, None, 0, 255, cv.NORM_MINMAX)
    img_c = cv.Canny(img_g2, 30, 70)
    return cv.merge((gray_img, img_c, img_d))
```

Promemoria: alcune funzioni che potrebbero essere utili

```
cvtColor(src, code[, dst[, dstCn]]) -> dst
GaussianBlur(src, ksize, sigmaX[, dst[, sigmaY[, borderType]]]) -> dst
normalize(src, dst[, alpha[, beta[, norm_type[, dtype[, mask]]]]) -> dst
Canny(image, threshold1, threshold2[, edges[, apertureSize[, L2gradient]]]) -> edges
```

Promemoria: alcune costanti che potrebbero essere utili

```
np.int16, np.uint8
cv.COLOR_BGR2GRAY, cv.COLOR_GRAY2BGR, cv.COLOR_BGR2RGB
cv.NORM_MINMAX
```